



LLM06 · Unbounded Consumption

OWASP LLM Top 10 (2026) · aka Denial of Wallet

Biggest mover in the 2026 rewrite

#10 → #6

2025 rank → 2026 rank



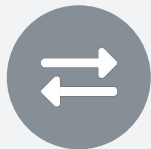
Why: agents

1 user message → many model + tool calls

Renamed scope

Model DoS → Unbounded Consumption

Requests are the wrong unit



1 request

60 req/min looks safe



100K tokens

huge input / output



Long reasoning

hidden thinking tokens



200 tool calls

agent fan-out

Cost scales with tokens + steps, not request count

Attack or accident: same bill



Malicious

- Stolen-key floods
- Overthinking prompts
- Poisoned fan-out



Accidental

- Retry / re-plan loops
- Context growing over 100 turns
- Bugs, no attacker

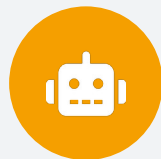
The vulnerability = the missing limit, not the intent

What attackers get out of it



Free compute

LLMjacking: stolen keys resold ·
~\$46K/day on the victim (Sysdig)



Freeloading

Your free chatbot = their unmetered LLM



Model extraction

Distill a copy from your outputs



Sabotage

Drain budget → spend cap trips →
outage

1-3 steal value · 4 causes damage

Stolen key $\neq$ LLM06 by itself



How they got in

Leaked key $\rightarrow$ secrets hygiene, LLM08, LLM04



How much damage

No cap $\rightarrow$ \$50 becomes \$46K/day $\rightarrow$ LLM06

Same split as LLM01 (gets control) vs LLM03 (what control can do)

Five consumption patterns



Request flood

stolen key, 50K calls
overnight



Tool fan-out

poisoned data →
recursive calls



Reasoning loop

"re-verify every step"



Context growth

turn 100 $\approx$ 50× turn 1



Extraction

query at scale → train
a copy

Defense = layered limits

Provider	Org / workspace spend cap	last-resort backstop
Gateway	Per-user token / \$ budget	one bad user $\neq$ everyone
Session	Compaction + session budget	context growth
Task	Max steps $\cdot$ \$ per task $\cdot$ timeout	runaway loops
Call	max_tokens $\cdot$ thinking budget	giant output, overthinking
Fan-out	Depth / width $\cdot$ shared budget	recursive spawning

The global-cap trap



One bucket

- 3:00 attacker drains the cap
- 3:01 all 10,000 users cut off
- **Sabotage wins**



Per-user budgets

- Provider sees only your keys
- Max granularity: workspace / project
- **Per-user limits live in your gateway**

Per-task: three different units



\$ / tokens

Bounds model spend

10 steps × 150K ctx = 1.5M tokens



Steps

Bounds actions + stuck
detection

*tool costs + side effects tokens don't
see*



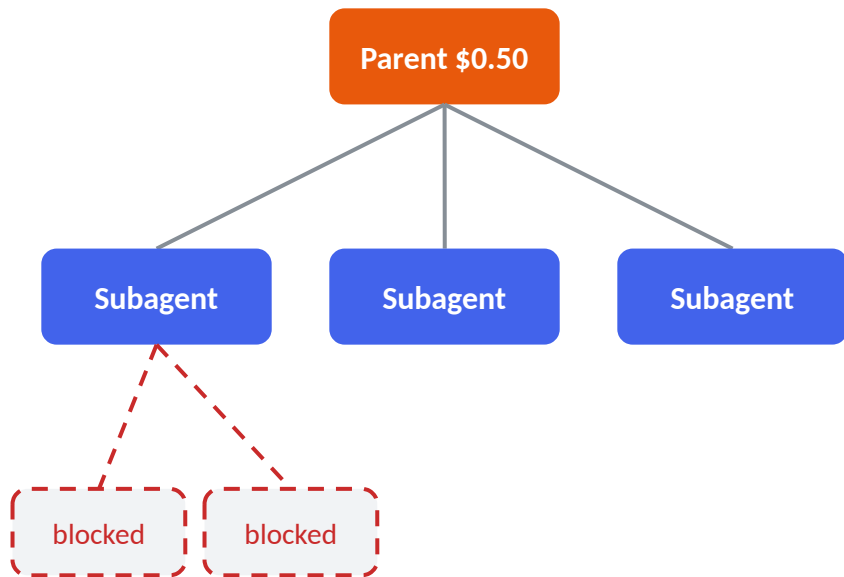
Wall clock

Bounds time

hung tools, open sandboxes

Per call: $\text{max_tokens (output)} + \text{thinking budget (budget_tokens / reasoning_effort)}$

Fan-out: cap the tree



Depth

levels of spawning (depth 1 = workers can't spawn)

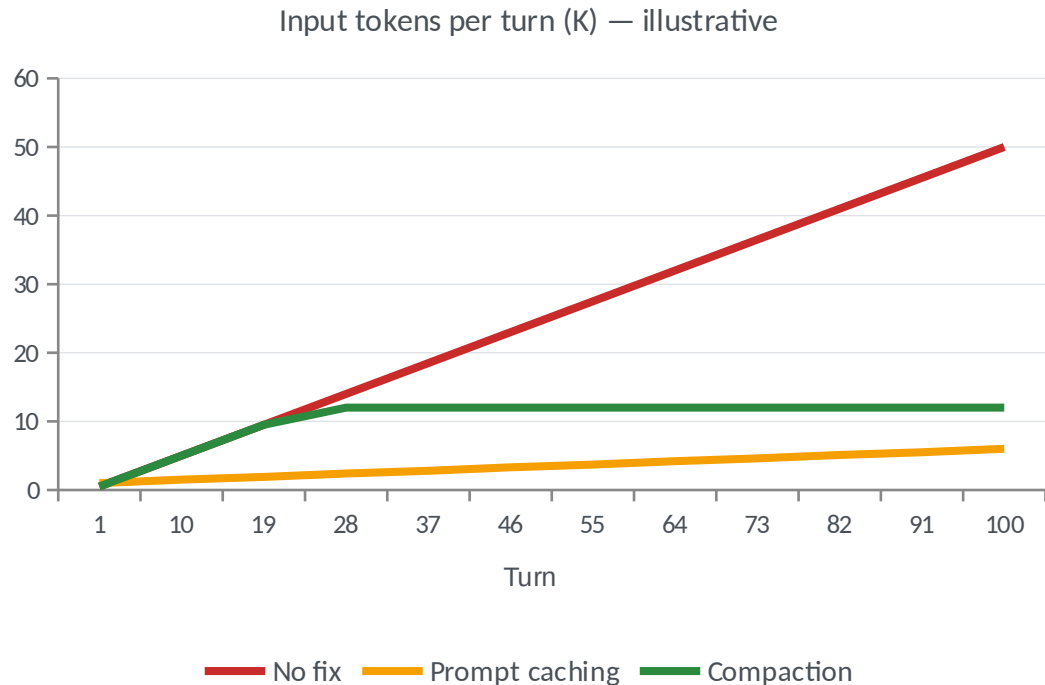
Width

max parallel children

Shared budget

the whole tree splits the parent's \$, not \$0.50 each

Long sessions: the context tax



Stateless model

full history resent each turn

per turn: linear • total: quadratic

Compaction

summarize old turns → bounds it

Caching

~10% price → cuts, doesn't bound

Model extraction at industrial scale

24,000

fraudulent accounts

16M+

exchanges to distill Claude



Sybil attack

- ~670 queries / account looks normal
- Proxy "hydra" networks, 20K+ accounts
- **Per-user budgets can't see it**

Defend where attackers can't multiply



Expensive identities

verification • payment • KYC



Cross-account detection

shared proxies, prompt templates, timing



Lower output value

no logprobs • hide raw chain-of-thought

A per-identity limit is only as strong as the cost of a new identity

Takeaways

- Meter tokens, \$, steps, and time — never just requests
- Limit at every layer; per-user lives in your stack
- Subagents share the parent's budget
- Compaction bounds sessions; caching only discounts
- Extraction is Sybil: defend across identities